

ALGORITMA DAN STRUKTUR DATA
LATIHAN PRAKTIKUM 2 STACK(TUMPUKAN)



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

LATIHAN

1. Lakukan operasi berikut :

- Jika karakter maka push ke stack
- Jika asterisk (*) maka pop dan cetak hasilnya.

Bagaimana output program dengan input seperti di bawah ini ?

E A S * Y * Q U E * * * S T * * * I O * N * * *

```
1. #include <stdio.h>
2. #include <string.h>
3. #define max 100
4.
5. typedef struct {
6.     char huruf[max];
7.     int count;
8. } stack;
9. void inisialisasi_stack(stack *);
10. char pop(stack *);
11. void push(stack *, char);
12. int main() {
13.     stack tumpukan;
14.     inisialisasi_stack(&tumpukan);
15.     char kata[] = "EAS*Y*QUE***ST***IO*N***";
16.     int panjang = strlen(kata);
17.
18.     printf ("Hasil program:\n");
19.     for (int i = 0; i < panjang - 1; i++)
20.     {
21.         if (kata[i] != '*') {
22.             push(&tumpukan, kata[i]);
23.         } else {
24.             char huruf = pop(&tumpukan);
25.             if (huruf != '\0') {
26.                 printf ("%c ", huruf);
27.             }
28.         }
29.     }
30. }
31. void inisialisasi_stack(stack *s) {
32.     s->count = 0;
33. }
34. void push(stack *s, char item) {
```

```

35.     if (s->count == max) {
36.         printf ("maaf stack sudah penuh");
37.     } else {
38.         s->huruf[s->count] = item;
39.         ++(s->count);
40.     }
41. }
42. char pop(stack *s) {
43.     if (s->count == 0) {
44.         printf ("stack masih kosong");
45.         return '\0';
46.     } else {
47.         --(s->count);
48.         return s->huruf[s->count];
49.     }
50. }

```

Output:

```

Hasil program:
S Y E U Q T S A O N I

```

Analisis Program:

Program ini menunjukkan cara kerja paling dasar dari struktur data stack (tumpukan) yang menggunakan prinsip LIFO (Last In First Out atau Terakhir Masuk, Pertama Keluar). Bayangkan sebuah tabung kosong; saat program membaca huruf (seperti E, A, S), huruf-huruf tersebut dimasukkan satu per satu ke dalam tabung. Namun, ketika program membaca simbol bintang (*), ini menjadi perintah untuk mengambil (mengeluarkan) huruf yang posisinya paling atas di dalam tabung tersebut dan mencetaknya ke layar. Karena huruf yang terakhir kali dimasukkan berada di posisi paling atas, maka huruf itulah yang akan keluar lebih dulu. Hal ini membuat urutan huruf yang tercetak menjadi sebagian terbalik dibandingkan urutan aslinya.

2. Buatlah sebuah program yang melakukan konversi dari bilangan desimal ke bilangan biner, octal, heksa dengan menggunakan stack

```

1. #include <stdio.h>
2. #define max 1000
3.
4. typedef int itemtype;
5.
6. typedef struct {
7.     itemtype item[max];

```

```
8.     int count;
9. } stack;
10.
11. void inisialisasi(stack *);
12. void push(stack *, itemtype);
13. itemtype pop(stack *);
14. void konversi(stack *, itemtype, int);
15.
16. int main() {
17.     stack tumpukan;
18.     itemtype bilangan;
19.     char jawab;
20.
21.     do {
22.         printf("Masukkan bilangan desimal: ");
23.         scanf("%d", &bilangan);
24.         printf("\n");
25.
26.         inisialisasi(&tumpukan);
27.         printf("Hasil konversi desimal %d ke biner : ",
bilangan);
28.         konversi(&tumpukan, bilangan, 2);
29.         printf("\n");
30.
31.         inisialisasi(&tumpukan);
32.         printf("Hasil konversi desimal %d ke octal : ",
bilangan);
33.         konversi(&tumpukan, bilangan, 8);
34.         printf("\n");
35.
36.         inisialisasi(&tumpukan);
37.         printf("Hasil konversi desimal %d ke hexa : ",
bilangan);
38.         konversi(&tumpukan, bilangan, 16);
39.         printf("\n");
40.
41.         printf("\nApakah ingin konversi lagi (y/n): ");
42.         scanf(" %c", &jawab);
43.
44.     } while (jawab == 'y' || jawab == 'Y');
45.
```

```

46.     return 0;
47.}
48.void inisialisasi(stack *s) {
49.    s->count = 0;
50.}
51.
52.void push(stack *s, itemtype x) {
53.    if (s->count == max) {
54.        printf("Stack penuh");
55.    } else {
56.        s->item[s->count] = x;
57.        s->count++;
58.    }
59.}
60.itemtype pop(stack *s) {
61.    if (s->count == 0) {
62.        return -1;
63.    } else {
64.        s->count--;
65.        return s->item[s->count];
66.    }
67.}
68.void konversi(stack *s, itemtype x, int basis) {
69.    int hasil;
70.    char digit[] = "0123456789ABCDEF";
71.
72.    if (x == 0) {
73.        printf("0");
74.        return;
75.    }
76.    while (x != 0) {
77.        push(s, x % basis);
78.        x = x / basis;
79.    }
80.    while (s->count != 0) {
81.        hasil = pop(s);
82.        printf("%c", digit[hasil]);
83.    }
84.}

```

Output:

```
Masukkan bilangan desimal: 123

Hasil konversi desimal 123 ke biner : 1111011
Hasil konversi desimal 123 ke octal : 173
Hasil konversi desimal 123 ke hexa : 7B

Apakah ingin konversi lagi (y/n): n
```

Analisis Program:

Pada program konversi bilangan, kita memanfaatkan stack untuk menampung sisa hasil bagi (modulo). Secara matematis, untuk mengubah bilangan desimal menjadi biner, angka tersebut harus dibagi 2 terus-menerus, dan sisa baginya dicatat. Sisa-sisa hasil pembagian ini dimasukkan berurutan ke dalam tumpukan. Setelah angka aslinya habis dibagi, program akan mengeluarkan isi tumpukan tersebut satu per satu. Karena sifat dasar stack adalah LIFO, sisa bagi yang dihitung paling akhir otomatis akan dicetak pertama kali. Urutan terbalik inilah yang memang dibutuhkan untuk membaca hasil konversi bilangan dengan benar. Khusus untuk bilangan heksadesimal (basis 16), program dibantu dengan susunan karakter khusus (0-9A-F) agar sisa hasil bagi bernilai 10 hingga 15 bisa langsung dicetak menjadi huruf A hingga F.

3. Buatlah sebuah program yang melakukan pembalikan terhadap kalimat dengan menggunakan stack**Contoh:**

Kalimat	:	Struktur Data
Hasil setelah dibalik :		ataD rutkurtS

```
1. #include <stdio.h>
2. #include <string.h>
3. #define max 1000
4.
5. typedef struct {
6.     char huruf[max];
7.     int count;
8. }stack;
9. void inisialisasi(stack *);
10. void push(stack *, char);
11. int pop(stack *);
12. void balikstring(stack *, char []);
```

```

13.int main() {
14.    stack tumpukan;
15.    char kalimat[max];
16.    int pjg;
17.
18.    puts ("PEMBALIK STRING");
19.    printf ("masukkan sebuah kalimat: ");
20.    fgets (kalimat, sizeof(kalimat), stdin);
21.    kalimat[strcspn(kalimat, "\n")] = '\0';
22.
23.    inisialisasi(&tumpukan);
24.    balikstring(&tumpukan, kalimat);
25.}
26.void inisialisasi(stack *s) {
27.    s->count = 0;
28.}
29.void push(stack *s, char x) {
30.    if (s->count == max) {
31.        printf ("stack sudah penuh");
32.    } else {
33.        s->huruf[s->count] = x;
34.        s->count++;
35.    }
36.}
37.int pop(stack *s) {
38.    if (s->count == 0) {
39.        printf ("string masih kosong");
40.        return '\0';
41.    } else {
42.        s->count--;
43.        return s->huruf[s->count];
44.    }
45.}
46.void balikstring(stack *s, char x[]) {
47.    int i = 0;
48.    while (x[i] != '\0') {
49.        push(s, x[i]);
50.        i++;
51.    }
52.    printf("Hasil: ");
53.    while (s->count != 0) {

```

```

54.         printf("%c", pop(s));
55.     }
56. }

```

Output:

```

PEMBALIK STRING
masukkan sebuah kalimat: Luthfi Bahrur Rozaq
Hasil: qazoR rurhaB ifhtuL

```

Analisis Program:

Program ini adalah contoh penerapan stack yang paling mudah dibayangkan. Tujuan program ini adalah membaca sebuah kalimat normal dan mencetaknya secara terbalik dari belakang ke depan. Caranya adalah dengan memasukkan setiap huruf dan spasi dari kalimat tersebut ke dalam stack secara berurutan. Ibarat menumpuk piring, piring yang ditaruh paling akhir pasti akan diambil pertama kali saat kita membongkar tumpukan tersebut. Saat program mengeluarkan semua isi stack sampai kosong, karakter terakhir dari kalimat secara otomatis akan tercetak lebih dulu di layar, sehingga susunan katanya menjadi terbalik secara sempurna.

4. Tentukan apakah sebuah kalimat yang diinputkan dalam program (dengan menggunakan stack) adalah sebuah palindrom atau bukan. Palindrom adalah kalimat yang jika dibaca dari depan dan dari belakang, maka bunyinya sama.

Contoh:

Kalimat : sugus
 Kalimat tersebut adalah palindrom

```

1. #include <stdio.h>
2. #include <string.h>
3. #define max 1000
4.
5. typedef struct {
6.     char huruf[max];
7.     int count;
8. }stack;
9. void inisialisasi(stack *);
10. void push(stack *, char);
11. char pop(stack *);
12. int validasi(stack *, char []);
13. int main() {
14.     stack tumpukan;
15.     char kalimat[max];
16.     char balik[max];

```

```

17.     int hasil, i = 0;
18.
19.     puts ("PEMBALIK STRING");
20.     printf ("masukkan sebuah kalimat: ");
21.     fgets (kalimat, sizeof(kalimat), stdin);
22.     kalimat[strlen(kalimat)] = '\0';
23.
24.     inisialisasi(&tumpukan);
25.     hasil = validasi(&tumpukan, balik);
26.
27.     printf("kalimat: %s\n", kalimat);
28.
29.     if (hasil == 1) {
30.         printf ("Kalimat terssebut adalah palindrom");
31.     } else {
32.         printf ("Kalimat terssebut bukan palindrom");
33.     }
34.}
35.void inisialisasi(stack *s) {
36.    s->count = 0;
37.}
38.void push(stack *s, char x) {
39.    if (s->count == max) {
40.        printf ("stack sudah penuh");
41.    } else {
42.        s->huruf[s->count] = x;
43.        s->count++;
44.    }
45.}
46.char pop(stack*s) {
47.    if (s->count == 0) {
48.        printf ("string masih kosong");
49.        return '\0';
50.    } else {
51.        s->count--;
52.        return s->huruf[s->count];
53.    }
54.}
55.int validasi(stack *s, char x[]) {
56.    int i = 0, j = 0;
57.    while (x[i] != '\0') {

```

```

58.     push(s, x[i]);
59.     i++;
60. }
61. while (x[j] != '\0') {
62.     if (x[j] != pop(s)) {
63.         return 0;
64.     }
65.     j++;
66. }
67.
68. return 1;
69. }

```

Output:

```

PEMBALIK STRING
masukkan sebuah kalimat: kasur rusak
kalimat: kasur rusak
Kalimat terssebut adalah palindrom

```

Analisis Program:

Kata atau kalimat palindrom adalah kata yang dibaca dari depan maupun dari belakang bunyinya tetap sama (contohnya kata "kasur rusak"). Program ini pada dasarnya meminjam logika pembalikan kalimat dari program sebelumnya. Huruf-huruf dari kalimat dimasukkan ke dalam stack, lalu dikeluarkan kembali satu per satu. Bedanya, huruf yang keluar dari stack tidak langsung dicetak ke layar, melainkan dicocokkan (dibandingkan) dengan huruf pada kalimat aslinya secara berurutan dari awal. Jika selama proses pencocokan ada satu saja huruf yang tidak sama, program akan langsung menyimpulkan bahwa kalimat itu bukan palindrom (menghasilkan nilai 0). Namun, jika semua huruf cocok hingga stack kosong, program akan menyimpulkan kalimat tersebut adalah palindrom (menghasilkan nilai 1).

5. Tambahkan implementasi Percobaan2 untuk mengecek apakah notasi infix yang diinputkan benar atau tidak.

Contoh :

Infix : 5 * (4 – 2

Output : Notasi infix salah, kurung tidak lengkap

Infix : 5 * (4 –)

Output : Notasi infix salah, operand tidak lengkap

Infix : 5 (4 – 2)

Output : Notasi infix salah, operator tidak lengkap

1. #include <stdio.h>
2. #include <string.h>

```

3. #include <ctype.h>
4.
5. #define MAXSTACK 100
6.
7. typedef char ItemType;
8. typedef struct {
9.     char item[MAXSTACK];
10.    int count;
11.} Stack;
12.
13. Stack tumpukan;
14.
15. void InitializeStack(Stack *);
16. int Empty(Stack *);
17. int Full(Stack *);
18. void Push(ItemType, Stack *);
19. ItemType Pop(Stack *);
20. int drjt(char);
21. void konversi_cetak(char []);
22. int cek_infix(char []);
23.
24. int main()
25. {
26.     char tampung[MAXSTACK], jawab;
27.
28.     puts("MENGUBAH NOTASI INFIX MENJADI POSTFIX");
29.     puts("DENGAN MEMANFAATKAN STRUKTUR STACK");
30.     do {
31.         InitializeStack(&tumpukan);
32.         fflush(stdin);
33.         puts("");
34.         printf("Masukan ekspresi dalam notasi infix : ");
35.         fgets(tampung, sizeof(tampung), stdin);
36.
37.         if (cek_infix(tampung)) {
38.             printf("Ungkapan postfixnya = ");
39.             konversi_cetak(tampung);
40.             puts("");
41.         }
42.
43.         fflush(stdin);

```

```

44.         printf("\nMau mencoba lagi (y/t) ? ");
45.         scanf(" %c", &jawab);
46.         while (getchar() != '\n');
47.     } while((jawab == 'y') || (jawab == 'Y'));
48.
49.     return 0;
50.}
51.
52.int cek_infix(char temp[])
53.{
54.    int i, pjg = strlen(temp);
55.    int kurung = 0;
56.    int prev_type = -1;
57.    char c;
58.
59.    for (i = 0; i < pjg; i++) {
60.        c = temp[i];
61.
62.        if (c == ' ' || c == '\n' || c == '\r') continue;
63.
64.        if (isdigit(c)) {
65.            if (prev_type == 3 || prev_type == 0) {
66.                printf("Output : Notasi infix salah, operator
tidak lengkap\n");
67.                return 0;
68.            }
69.            prev_type = 0;
70.        }
71.        else if (c == '+' || c == '-' || c == '*' || c == '/'
|| c == '^') {
72.            if (prev_type == -1 || prev_type == 2) {
73.                printf("Output : Notasi infix salah, operand
tidak lengkap\n");
74.                return 0;
75.            }
76.            prev_type = 1;
77.        }
78.        else if (c == '(') {
79.            if (prev_type == 0 || prev_type == 3) {
80.                printf("Output : Notasi infix salah, operator
tidak lengkap\n");

```

```

81.         return 0;
82.     }
83.     prev_type = 2;
84.     kurung++;
85. }
86.     else if (c == ')') {
87.         if (prev_type == 1 || prev_type == 2) {
88.             printf("Output : Notasi infix salah, operand
tidak lengkap\n");
89.             return 0;
90.         }
91.         prev_type = 3;
92.         kurung--;
93.
94.         if (kurung < 0) {
95.             printf("Output : Notasi infix salah, kurung
tidak lengkap\n");
96.             return 0;
97.         }
98.     }
99. }
100.
101.     if (kurung != 0) {
102.         printf("Output : Notasi infix salah, kurung
tidak lengkap\n");
103.         return 0;
104.     }
105.
106.     if (prev_type == 1) {
107.         printf("Output : Notasi infix salah, operand
tidak lengkap\n");
108.         return 0;
109.     }
110.
111.     return 1;
112. }
113.
114. void InitializeStack(Stack *S)
115. {
116.     S->count = 0;
117. }

```

```

118.
119.     int Empty(Stack *S)
120.     {
121.         return (S->count == 0);
122.     }
123.
124.     int Full(Stack *S)
125.     {
126.         return (S->count == MAXSTACK);
127.     }
128.
129.     void Push(ItemType x, Stack *S)
130.     {
131.         if (Full(S))
132.             printf("Stack penuh! Data tidak dapat masuk!");
133.         else {
134.             ++(S->count);
135.             S->item[S->count] = x;
136.         }
137.     }
138.
139.     ItemType Pop(Stack *S)
140.     {
141.         ItemType x;
142.         if (Empty(S)) {
143.             return 0;
144.         }
145.         else {
146.             x = (S->item[S->count]);
147.             --(S->count);
148.             return x;
149.         }
150.     }
151.
152.     int drjt(char x)
153.     {
154.         if (x == '(')
155.             return 0;
156.         else if((x == '+' ) || (x == '-'))
157.             return 1;
158.         else if ((x == '*' ) || (x == '/'))

```

```

159.         return 2;
160.     else if (x == '^')
161.         return 3;
162.     else
163.         return -1;
164. }
165.
166. void konversi_cetak(char temp[])
167. {
168.     int i, pjg, valid = 1;
169.     char kar, smtr;
170.
171.     pjg = strlen(temp) - 1;
172.
173.     for (i = 0; i < pjg; i++)
174.     {
175.         kar = temp[i];
176.
177.         switch (kar) {
178.             case '(':
179.                 Push(kar, &tumpukan);
180.                 break;
181.
182.             case '0': case '1': case '2': case '3': case
183. '4':
184.             case '5': case '6': case '7': case '8': case
185. '9':
186.                 printf("%c", kar);
187.                 break;
188.
189.             case '+': case '-': case '*': case '/': case
190. '^':
191.                 if((Empty(&tumpukan)) || ((drjt(kar) >
192. drjt(tumpukan.item[tumpukan.count]))))
193.                     Push(kar, &tumpukan);
194.                 else {
195.                     do {
196.                         smtr = Pop(&tumpukan);
197.                         printf("%c", smtr);
198.                     } while (drjt(kar) <=
199. drjt(tumpukan.item[tumpukan.count]) && !Empty(&tumpukan));

```

```

195.             Push(kar, &tumpukan);
196.         }
197.         break;
198.
199.         case ')':
200.             while (tumpukan.item[tumpukan.count] !=
    '(') {
201.                 smtr = Pop(&tumpukan);
202.                 printf("%c", smtr);
203.             }
204.             Pop(&tumpukan);
205.             break;
206.
207.         case ' ': case '\n': case '\r':
208.             break;
209.
210.         default:
211.             valid = 0;
212.             break;
213.     }
214. }
215.
216. if ((valid != 0) && (!Empty(&tumpukan)))
217. {
218.     while (!Empty(&tumpukan)) {
219.         smtr = Pop(&tumpukan);
220.         printf("%c", smtr);
221.     }
222. }
223. }

```

Output:

DENGAN MEMANFAATKAN STRUKTUR STACK

Masukan ekspresi dalam notasi infix : 5 * (4 - 2

Output : Notasi infix salah, kurung tidak lengkap

Mau mencoba lagi (y/t) ? t

MENGUBAH NOTASI INFIX MENJADI POSTFIX
DENGAN MEMANFAATKAN STRUKTUR STACK

Masukan ekspresi dalam notasi infix : $5 * (4 - 2)$
Ungkapan postfixnya = $542-*$

Mau mencoba lagi (y/t) ? t

Analisis Program:

Cara manusia menulis rumus matematika (disebut format infix) seringkali membingungkan komputer karena adanya prioritas perhitungan seperti perkalian yang harus dihitung lebih dulu daripada penjumlahan, ditambah lagi dengan adanya tanda kurung. Program ini menggunakan stack untuk mengatur ulang rumus tersebut menjadi format postfix yang lebih ramah komputer. Sebelum diubah, program akan mengecek dulu apakah penulisan rumusnya sudah benar (kurung buka dan tutupnya lengkap, serta posisi angkanya masuk akal). Saat proses konversi berjalan, angka akan langsung dicetak, sedangkan simbol operasi matematika (+, -, *, /) akan dititipkan sementara ke dalam stack. Simbol yang punya prioritas lebih tinggi akan dipaksa keluar lebih dulu dari tumpukan, memastikan aturan perkalian dan pembagian dikerjakan terlebih dahulu sesuai aturan matematika standar.

6. Tambahkan implementasi Percobaan2 hingga mendapatkan hasil penghitungannya.

Contoh:

Infix: $5 * (4 - 2)$
Postfix: $542-*$
Hasil: 10

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <ctype.h>
4. #include <math.h>
5.
6. #define MAXSTACK 100
7.
8. typedef char ItemType;
9. typedef struct {
10.     char item[MAXSTACK];
11.     int count;
12. } Stack;
13.
14. Stack tumpukan;
15.
16. void InitializeStack(Stack *);
17. int Empty(Stack *);
```

```

18.int Full(Stack *);
19.void Push(ItemType, Stack *);
20.ItemType Pop(Stack *);
21.int drjt(char);
22.void konversi_cetak(char[], char[]);
23.
24.typedef struct {
25.    int item[MAXSTACK];
26.    int count;
27.} IntStack;
28.
29.void InitIntStack(IntStack *S) { S->count = 0; }
30.
31.void PushInt(int x, IntStack *S) {
32.    if (S->count < MAXSTACK) {
33.        ++(S->count);
34.        S->item[S->count] = x;
35.    }
36.}
37.
38.int PopInt(IntStack *S) {
39.    if (S->count == 0) return 0;
40.    int x = S->item[S->count];
41.    --(S->count);
42.    return x;
43.}
44.
45.int hitung_postfix(char[]);
46.
47.int main()
48.{
49.    char tampung[MAXSTACK], jawab;
50.    char postfix[MAXSTACK];
51.
52.    puts("MENGUBAH NOTASI INFIX MENJADI POSTFIX & MENGHITUNG
HASIL");
53.    puts("DENGAN MEMANFAATKAN STRUKTUR STACK");
54.    do {
55.        InitializeStack(&tumpukan);
56.        memset(postfix, 0, sizeof(postfix));
57.        fflush(stdin);

```

```

58.
59.     puts("");
60.     printf("Masukan ekspresi dalam notasi infix : ");
61.     fgets(tampung, sizeof(tampung), stdin);
62.
63.     tampung[strcspn(tampung, "\n")] = 0;
64.
65.     printf("postfix : ");
66.     konversi_cetak(tampung, postfix);
67.     puts("");
68.
69.     int hasil = hitung_postfix(postfix);
70.     printf("hasil : %d\n", hasil);
71.
72.     fflush(stdin);
73.     printf("\nMau mencoba lagi (y/t) ? ");
74.     scanf(" %c", &jawab);
75.     while (getchar() != '\n');
76. } while((jawab == 'y') || (jawab == 'Y'));
77.
78.     return 0;
79.}
80.
81.void InitializeStack(Stack *S) {
82.    S->count = 0;
83.}
84.
85.int Empty(Stack *S) {
86.    return (S->count == 0);
87.}
88.
89.int Full(Stack *S) {
90.    return (S->count == MAXSTACK);
91.}
92.
93.void Push(ItemType x, Stack *S) {
94.    if (Full(S))
95.        printf("Stack penuh! Data tidak dapat masuk!");
96.    else {
97.        ++(S->count);
98.        S->item[S->count] = x;

```

```

99.     }
100.    }
101.
102.    ItemType Pop(Stack *S) {
103.        ItemType x;
104.        if (Empty(S)) {
105.            return 0;
106.        }
107.        else {
108.            x = (S->item[S->count]);
109.            --(S->count);
110.            return x;
111.        }
112.    }
113.
114.    int drjt(char x) {
115.        if (x == '(') return 0;
116.        else if((x == '+' ) || (x == '-')) return 1;
117.        else if ((x == '*' ) || (x == '/')) return 2;
118.        else if (x == '^') return 3;
119.        else return -1;
120.    }
121.
122.    void konversi_cetak(char temp[], char postfix[]) {
123.        int i, pjg, valid = 1, idx = 0;
124.        char kar, smtr;
125.
126.        pjg = strlen(temp);
127.
128.        for (i = 0; i < pjg; i++) {
129.            kar = temp[i];
130.
131.            switch (kar) {
132.                case '(':
133.                    Push(kar, &tumpukan);
134.                    break;
135.
136.                case '0': case '1': case '2': case '3': case
137.                    '4':
138.                case '5': case '6': case '7': case '8': case
139.                    '9':

```

```

138.         printf("%c", kar);
139.         postfix[idx++] = kar;
140.         break;
141.
142.         case '+': case '-': case '*': case '/': case
    '^':
143.             if((Empty(&tumpukan)) || ((drjt(kar) >
    drjt(tumpukan.item[tumpukan.count]))))
144.                 Push(kar, &tumpukan);
145.             else {
146.                 do {
147.                     smtr = Pop(&tumpukan);
148.                     printf("%c", smtr);
149.                     postfix[idx++] = smtr;
150.                 } while (drjt(kar) <=
    drjt(tumpukan.item[tumpukan.count]) && !Empty(&tumpukan));
151.                 Push(kar, &tumpukan);
152.             }
153.             break;
154.
155.         case ')':
156.             while (tumpukan.item[tumpukan.count] !=
    '(') {
157.                 smtr = Pop(&tumpukan);
158.                 printf("%c", smtr);
159.                 postfix[idx++] = smtr;
160.             }
161.             Pop(&tumpukan);
162.             break;
163.
164.         case ' ': case '\n': case '\r':
165.             break;
166.
167.         default:
168.             valid = 0;
169.             break;
170.     }
171. }
172.
173. if ((valid != 0) && (!Empty(&tumpukan))) {
174.     while (!Empty(&tumpukan)) {

```

```

175.         smtr = Pop(&tumpukan);
176.         printf("%c", smtr);
177.         postfix[idx++] = smtr;
178.     }
179. }
180.
181.     postfix[idx] = '\0';
182. }
183.
184. int hitung_postfix(char postfix[]) {
185.     IntStack evalStack;
186.     InitIntStack(&evalStack);
187.
188.     int i, len = strlen(postfix);
189.     for (i = 0; i < len; i++) {
190.         char c = postfix[i];
191.
192.         if (isdigit(c)) {
193.             PushInt(c - '0', &evalStack);
194.         }
195.         else {
196.             int angkaKanan = PopInt(&evalStack);
197.             int angkaKiri = PopInt(&evalStack);
198.             int res = 0;
199.
200.             switch(c) {
201.                 case '+': res = angkaKiri + angkaKanan;
202.                     break;
203.                 case '-': res = angkaKiri - angkaKanan;
204.                     break;
205.                 case '*': res = angkaKiri * angkaKanan;
206.                     break;
207.                 case '/': res = angkaKiri / angkaKanan;
208.                     break;
209.                 case '^': res = (int)pow(angkaKiri,
210.                     angkaKanan); break;
211.             }
212.             PushInt(res, &evalStack);
213.         }
214.     }
215. }

```

```

211.
212.         return PopInt(&evalStack);
213.     }

```

Output:

```

MENGUBAH NOTASI INFIX MENJADI POSTFIX & MENGHITUNG HASIL
DENGAN MEMANFAATKAN STRUKTUR STACK

```

```

Masukan ekspresi dalam notasi infix   : 5 * ( 4 - 2 )
postfix : 542-*
hasil   : 10

```

```

Mau mencoba lagi (y/t) ? t

```

Analisis Program:

Setelah rumus matematika berhasil diubah menjadi bentuk postfix pada program sebelumnya, program ini bertugas untuk menghitung hasil akhirnya. Berbeda dengan stack sebelumnya yang menyimpan karakter atau simbol, stack pada program ini dibuat khusus untuk menyimpan angka (tipe data integer). Cara kerjanya sangat terstruktur: jika program membaca angka, angka itu langsung dimasukkan ke dalam tumpukan. Jika program membaca simbol operasi hitung (seperti tambah atau kali), program akan mengambil dua angka terakhir dari bagian atas tumpukan, menghitung kedua angka tersebut sesuai simbolnya, dan memasukkan hasil perhitungannya kembali ke dalam tumpukan. Proses ini diulang terus sampai rumus habis terbaca. Pada akhirnya, satu-satunya angka yang tersisa di dalam tumpukan adalah jawaban atau hasil akhir dari perhitungan matematika tersebut.

Kesimpulan

Berdasarkan serangkaian percobaan dan praktikum mengenai struktur data Stack (Tumpukan) yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Struktur data Stack sangat bergantung pada prinsip LIFO (Last In First Out), di mana data yang paling terakhir dimasukkan akan menjadi data yang pertama kali dikeluarkan. Prinsip yang sederhana ini terbukti sangat efektif untuk menyelesaikan masalah pemrograman yang membutuhkan proses penelusuran mundur atau pembalikan urutan, seperti membalik kalimat, membaca konversi sisa hasil bagi, hingga menyusun ulang perhitungan matematika.
2. Dalam bahasa pemrograman C, stack paling mudah dibangun dengan menggunakan array (larik) satu dimensi sebagai tempat penyimpanan datanya. Untuk mengontrol proses masuk keluarnya data, kita cukup menggunakan satu variabel bantuan (sering disebut variabel count atau top) yang bertugas mengingat posisi indeks teratas dari tumpukan tersebut.

3. Kunci utama dari keamanan data pada struktur stack adalah kepatuhan dalam menggunakan fungsi Push (untuk menambah data) dan Pop (untuk mengambil data). Dengan selalu menggunakan kedua fungsi ini, kita mencegah terjadinya modifikasi data secara sembarangan di tengah-tengah array, sehingga urutan data tetap konsisten dan tidak rusak.
4. Struktur stack sangat fleksibel dan dapat dikombinasikan. Untuk menyelesaikan masalah yang rumit seperti membuat kalkulator perhitungan matematika, kita bisa menggunakan lebih dari satu stack di dalam satu program. Misalnya, satu stack bertipe karakter digunakan untuk mengatur urutan simbol matematika, sementara stack lain bertipe angka (integer) digunakan untuk menyimpan nilai dan menghitung hasil akhirnya secara bersamaan.